does not "remember" this sequence v, it may explore a completely different sequence, which could potentially lead to sleep set blocking. To prevent such a situation, we replace the backtrack set by a so-called *wakeup tree*. The wakeup tree contains initial fragments of the sequences that are to be explored after E'. Each fragment guarantees that no sleep set blocking will be encountered during the exploration.

6.1 Formal Definition

Let us define an *ordered tree* as a pair $\langle B, \prec \rangle$, where B (the set of *nodes*) is a finite prefix-closed set of sequences of processes, with the empty sequence $\langle\rangle$ being the root. The children of a node w, of form $w.p$ for some set of processes p, are ordered by $\prec$. In $\langle B, \prec \rangle$, such an ordering between children has been extended to the total order $\prec$ on B by letting $\prec$ be the induced post-order relation between the nodes in B. This means that if the children $w.p_1$ and $w.p_2$ are ordered as $w.p_1 \prec w.p_2$, then $w.p_1 \prec w.p_2 \prec w$ in the induced post-order. An example of the ordering of sequences is given in Fig. 7.

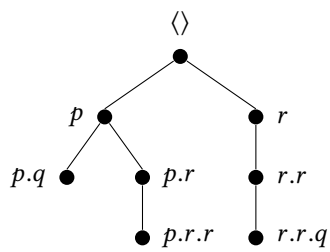

Fig. 7. A wakeup tree. The order of exploration $\prec$ is $p.q$, followed by $p.r.r$, $p.r$, and so on until we finish at $\langle\rangle$.

Definition 6.1 (Wakeup Tree). Let E be an execution sequence, and P be a set of processes. A *wakeup tree after* $\langle E, P \rangle$ is an ordered tree $\langle B, \prec \rangle$, such that the following properties hold

(1) $WI_{[E]}(w) \cap P = \emptyset$ whenever w is a leaf of B;
(2) whenever $u.p$ and $u.w$ are nodes in B with $u.p \prec u.w$, and $u.w$ is a leaf, then $p \notin WI_{[E.u]}(w)$.

Intuitively, a wakeup tree after $\langle E, P \rangle$ is intended to consist of initial fragments of sequences that should be explored after E to avoid sleep set blocking, when P is the current sleep set at E. To see this, note that if $q \in P$, then (by the way sleep sets are handled) q cannot be in $I_{[E]}(w)$ for any sequence w that is explored after E. If, however, $E \vdash q \diamond w$, then q is still in the sleep set at $E.w$ and may never be removed. To prevent this, we therefore require $q \notin WI_{[E]}(w)$, which is the same as Property 1, i.e., $WI_{[E]}(w) \cap P = \emptyset$. Property 2 implies that when a process p is added to the sleep set at $E.u$, after exploring $E.u.p$, then by the same reasoning as above, it will have been removed from the sleep set when we reach $E.u.w$.

The *empty* wakeup tree is the tree $\langle \{\langle\rangle\}, \emptyset \rangle$, which consists only of the root $\langle\rangle$. We state a useful property of wakeup trees.

Lemma 6.2. *If $\langle B, \prec \rangle$ is a wakeup tree after $\langle E, P \rangle$ and $w, w' \in B$ and w is a leaf which satisfies $w' \sim_{[E]} w$, then $w \preceq w'$.*

The lemma states that any leaf w is the smallest (w.r.t. $\prec$) node in the tree which is consistent with w after E.